

Exercises on Microservices with Spring Boot 3.0

1. Build a User and Order Management System

Problem: Create two microservices:

User Service to manage users.

Order Service to manage orders placed by users.

Requirements:

Use **REST APIs**.

Communicate between services using **WebClient (Spring WebFlux)** or

OpenFeign.

Store data in **MySQL** or **PostgreSQL**.

@Entity

```
public class User {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private String email;
```

```
}
```

```
public interface UserRepository extends JpaRepository<User, Long> {
```

```
}
```

```
@RestController
```

```
@RequestMapping("/users")
```

```
public class UserController {
```

```
    @Autowired
```

```
    private UserRepository repository;
```

```
    @GetMapping
```

```
    public List<User> getUsers() {
```

```
        return repository.findAll();
```

```
}
```

```
    @PostMapping
```

```
    public User addUser(@RequestBody User user) {
```

```
        return repository.save(user);
```

```
}
```

```
@Entity
```

```
public class Orders {
```

```
    @Id
```

```

    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private String product;
    private int quantity;
}
@FeignClient(name = "user-service", url = "http://localhost:8081")
public interface UserClient {

    @GetMapping("/users/{id}")
    User getUser(@PathVariable Long id);

}
@Service
public class OrderService {

    @Autowired
    private UserClient client;

    public User getUser(Long id) {
        return client.getUser(id);
    }
}

```

2. Inventory Management System with Service Discovery

Problem: Create:

Product Service: Manage products and stock.

Inventory Service: Track stock levels for each product.

Requirements:

Use **Spring Cloud Netflix Eureka** for **service discovery**.

Implement **centralized configuration** using Spring Cloud Config Server.

@SpringBootApplication

@EnableEurekaServer

```

public class EurekaServerApplication {

    public static void main(String[] args) {

        SpringApplication.run(EurekaServerApplication.class,args);

    }
}

```

```
@EnableConfigServer
@SpringBootApplication
public class ConfigServerApplication {
}
```

3. Implement an API Gateway

Problem: Create an **API Gateway** to route requests to:

Customer Service

Billing Service

Requirements:

Use **Spring Cloud Gateway**.

Implement **rate limiting**, **caching**, and **path rewriting**. **4. Resilient Microservices with Circuit Breaker**

Problem: A **Payment Service** calls a slow third-party API.

Requirements:

Implement **Circuit Breaker** and **fallback logic** using **Resilience4j**.

Log and monitor fallback events.

server:

port: 8080

spring:

cloud:

gateway:

routes:

- id: customer-service

uri: http://localhost:8081

predicates:

- Path=/customer/**

- id: billing-service

uri: http://localhost:8082

predicates:

- Path=/billing/**

filters:

- RequestRateLimiter=10,20

4. Resilient Microservices with Circuit Breaker

Problem: A **Payment Service** calls a slow third-party API.

Requirements:

Implement **Circuit Breaker** and **fallback logic** using **Resilience4j**.

Log and monitor fallback events.

<dependency>

<groupId>io.github.resilience4j</groupId>

```

    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
@Service
public class PaymentService {

    @CircuitBreaker(name = "paymentService",
        fallbackMethod = "fallback")

    public String processPayment() {

        throw new RuntimeException();

    }

    public String fallback(Exception e) {

        return "Payment Service is unavailable.";

    }
}
@RestController
public class PaymentController {

    @Autowired
    private PaymentService service;

    @GetMapping("/pay")
    public String pay() {

        return service.processPayment();

    }
}
public String fallback(Exception ex) {

    System.out.println("Fallback executed: " + ex.getMessage());

    return "Payment Service is temporarily unavailable.";

}

```